

KoreDB: Comprehensive Technical Documentation

1. Introduction

KoreDB is an AI-Native, Multi-Model Database Engine designed for modern Android applications. It natively supports three core database paradigms—Document Store, Vector Database, and Graph Database—all unified within a single, lightweight engine.

Unlike traditional databases like SQLite which were designed for spinning disks in the year 2000, KoreDB is built from the ground up using a Log-Structured Merge-tree (LSM) architecture. This makes it highly optimized for modern flash storage devices, high-concurrency operations using Kotlin Coroutines, on-device AI workloads, and complex relational data structures.

This documentation covers KoreDB's capabilities, implementation details, and provides a comprehensive guide for developers looking to integrate and leverage KoreDB in their applications.

2. Core Architecture & Implementation Details

KoreDB is 100% pure Kotlin. It has zero JNI overhead and avoids compiling traditional C/C++ database libraries like SQLite.

2.1 Log-Structured Merge-tree (LSM) Engine

KoreDB utilizes an LSM-tree architecture, similar to what powers distributed databases like Bigtable, Cassandra, and RocksDB, but optimized for embedded mobile use.

The Write Path ($O(1)$ Complexity)

1. **CommitLog (WAL):** Every mutation is immediately appended to a sequential log on disk (Write-Ahead Logging). This is crash-safe and includes CRC32 checksums.
2. **MemTable:** Once the WAL is successfully written, the update is applied to an in-memory sorted tree structure. The write operation returns instantly.

The Read Path

1. **Object Cache:** An LRU cache (holding up to 65K entries) avoids expensive I/O and JSON deserialization by serving recently accessed objects.
2. **MemTable:** If the object isn't in the cache, the engine checks the in-memory MemTable.
3. **Bloom Filter:** Before hitting the disk, a probabilistic Bloom Filter is checked. This avoids 99% of unnecessary disk reads if the key doesn't exist.
4. **SSTables:** Finally, a binary search is performed on disk-backed Sorted String Tables (SSTables) using a sparse index for $O(\log N)$ lookup times.

2.2 Advanced Secondary Indexing

Traditional secondary indexes often introduce complexity in embedded environments. KoreDB implements:

- **Per-entry covering indices** and **write-time anti-entry (tombstone) logic**.
- **$O(\log N + K)$ complexity** for secondary lookups, matching primary search parity.
- Reverse-pointer staleness detection eliminates stale results instantly upon lookup or compaction.

2.3 Hybrid HNSW Vector Indexing

KoreDB implements Hierarchical Navigable Small World (HNSW) graphs for Approximate Nearest Neighbor (ANN) search.

- **Background Hydration:** HNSW graphs are memory-heavy and can slow app startup. KoreDB handles HNSW hydration on a background thread. While hydrating, hybrid search falls back to flat scans automatically.
 - **Scalar Quantization (SQ8):** Memory consumption is reduced by 4x via SQ8 quantization with less than a 1% drop in recall accuracy.
 - **SIMD-Friendly Distances:** Four distance metrics (Cosine, Euclidean, Inner Product, Manhattan) are optimized using 4x loop unrolling for max performance on ARM processors.
-

3. Developer Guide

3.1 Setup and Initialization

Add Dependencies:

```
dependencies {  
    implementation("io.github.raipankaj:koredb:0.0.7")  
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.8.0")  
}
```

Initialize Database: KoreDB supports lazy initialization and background warmup for optimized startup times.

```
class MyApplication : Application() {  
    val database: KoreDatabase by lazy {  
        KoreAndroid.create(this, "my_db")  
    }  
  
    override fun onCreate() {  
        super.onCreate()  
        // Trigger background lazy initialization  
        CoroutineScope(Dispatchers.IO).launch {  
            database.engine  
        }  
    }  
}
```

3.2 Document Collections (Pillar 1)

Document collections provide a schema-less (but Kotlin-typed) interface for storing objects.

```
@Serializable  
data class User(val id: String, val name: String, val age: Int)  
  
val users = database.collection<User>("users")  
  
// Inserting Data  
users.insert("u1", User("u1", "Alice", 28))  
  
// Querying with DSL  
val adults = users.query()
```

```

        .where("age") { it.toInt() >= 18 }
        .sortBy("age") { it.toInt() }
        .limit(10)
        .execute()

// Reactive Observables
users.observeById("u1").collect { user ->
    println("User updated: $user")
}

```

Key Features:

- Range scans, secondary indexes (`createIndex`), and aggregations (`sum` , `avg` , `min` , `max`).
- Partial updates without rewriting entire objects (`updateFields`).

3.3 Vector Database (Pillar 2)

Perfect for local LLMs and AI features, the Vector Database stores high-dimensional embeddings and performs semantic search.

```

val embeddings = database.vectorCollection("embeddings") {
    dimensions = 768
    metric = DistanceMetric.COSINE
    quantization = true
}

// Inserting Vector with Metadata
embeddings.insert("doc_1", floatArrayOf(...), mapOf("category" to "ai"))

// Hybrid Semantic Search
val results = embeddings.search(queryVector, limit = 5) {
    where("category", eq("ai"))
}

```

3.4 Graph Database (Pillar 3)

The Graph engine manages Nodes and Edges for modeling complex relationships (e.g., social networks, knowledge graphs).

```

val graph = database.graph()

graph.transaction {
    putNode(Node("u1", labels = setOf("Person"), properties = mapOf("name" to "Alice")))
    putNode(Node("u2", labels = setOf("Person"), properties = mapOf("name" to "Bob")))
    putEdge(Edge("u1", "u2", "KNOWS"))
}

// Complex Pathfinding
val path = GraphAlgorithms.aStarPath(graph, "u1", "u2", "KNOWS") { n1, n2 ->
    // heuristic function
}

```

```
0.0  
}
```

Graph Algorithms Included:

- Dijkstra, A* Pathfinding, Variable Length Paths.
- Centrality measures (PageRank, Degree Centrality).
- Community Detection (Louvain).

3.5 The Unified Graph-Vector Bridge

KoreDB allows developers to combine the semantic search capabilities of Vectors with the relationship mapping of Graphs. This natively forms the basis of **GraphRAG (Graph Retrieval-Augmented Generation)**, providing significantly better context for LLMs compared to traditional vector-only retrieval.

1. Full GraphRAG Pipeline

The standard `graphRAGQuery` finds semantic seeds via Vector search, then traverses the Graph for expanded structural context.

```
val bridge = database.graphVectorBridge(embeddings)  
  
val graphRagContext = bridge.graphRAGQuery(  
  query = promptEmbedding,  
  initialLimit = 5,           // Vector Search: Find top 5 relevant seed nodes  
  edgeType = "DEPENDS_ON",   // Graph Expansion: Traverse specific relationships  
  maxHops = 2,               // Graph Expansion: Pull context up to 2 levels deep  
  finalLimit = 10            // Rerank: Return the 10 best contextual nodes  
)
```

2. Vector-First Query (Semantic Filtered by Structure)

This allows you to find vectors similar to a query, and then filter out the results that do not match a specific graph topology requirement.

```
// Find similar items, then filter them by the user's graph network  
val results = bridge.vectorSearch(queryEmbedding, limit = 50)  
  .filterByGraph { productId ->  
    graph.getOutboundTargetIds(productId, "MADE_BY")  
      .any { it in userFollowedBrands }  
  }
```

3. Graph-First Traversal (Structure Reranked by Semantic)

This allows you to first traverse the graph to get a deterministic set of connected nodes, and then rank those nodes based on semantic similarity to the user's query.

```
// Traverse 2 hops from the user to find related nodes, then rerank those by  
semantic relevance  
val ranked = bridge.graphTraversal("user_123", "PURCHASED", hops = 2)  
  .rerankByVector(queryEmbedding)  
  .take(10)
```

4. Property-Based Graph Start to Vector Rerank

You can initiate a graph lookup via a standard property index, and then rerank those matching nodes via the vector engine.

```
val results = bridge.graphQuery("Product", "category", "shoes")
    .rerankByVector(queryEmbedding)
    .take(10)
```

4. Conclusion

KoreDB fundamentally shifts how developers build on-device intelligence. By unifying Documents, Vectors, and Graphs under a single, highly-optimized Kotlin engine, it eliminates complex multi-database architectures and paves the way for advanced local AI applications.